

Implementation Steps

- Modify `opendfx_shell` block design
 - modify `opendfx_shell.tcl` to delete unwanted RMs
 - connect the ports using Vivado GUI
 - export the BD tcl script with *`write_bd_tcl`* command
 - modify address segments using tcl (using GUI will fail)
 - `assign_bd_address`
 - `set_property APERTURES`
 - write back to `opendfx_shell.tcl`
- Modify synthesis constraints
 - define pblocks in `physical_constraints.xdc`
- Modify RMs
 - support addressing for `RP_0`, `RP_1`, and `RP_2`
 - add new RM back to `opendfx_shell.tcl` for implementation
- Run Synthesis & Implementation, whether using tcl or Vivado GUI
- Modify Applications (the lib and the apps)

Implementation Notes

- There are only 4 PL Fabric Clocks, 2 of them are used by the shell (PL0) and the VCU (PL1, may be deleted), so there are 2 or 3 clocks for 3 RPs.

Re-customize IP

Zynq UltraScale+ MPSoC (3.4)

Documentation Presets IP Location

Page Navigator

- Switch To Advanced Mode
- PS UltraScale+ Block Design
- I/O Configuration
- Clock Configuration
- DDR Configuration
- PS-PL Configuration

Clock Configuration

Input Clocks Output Clocks

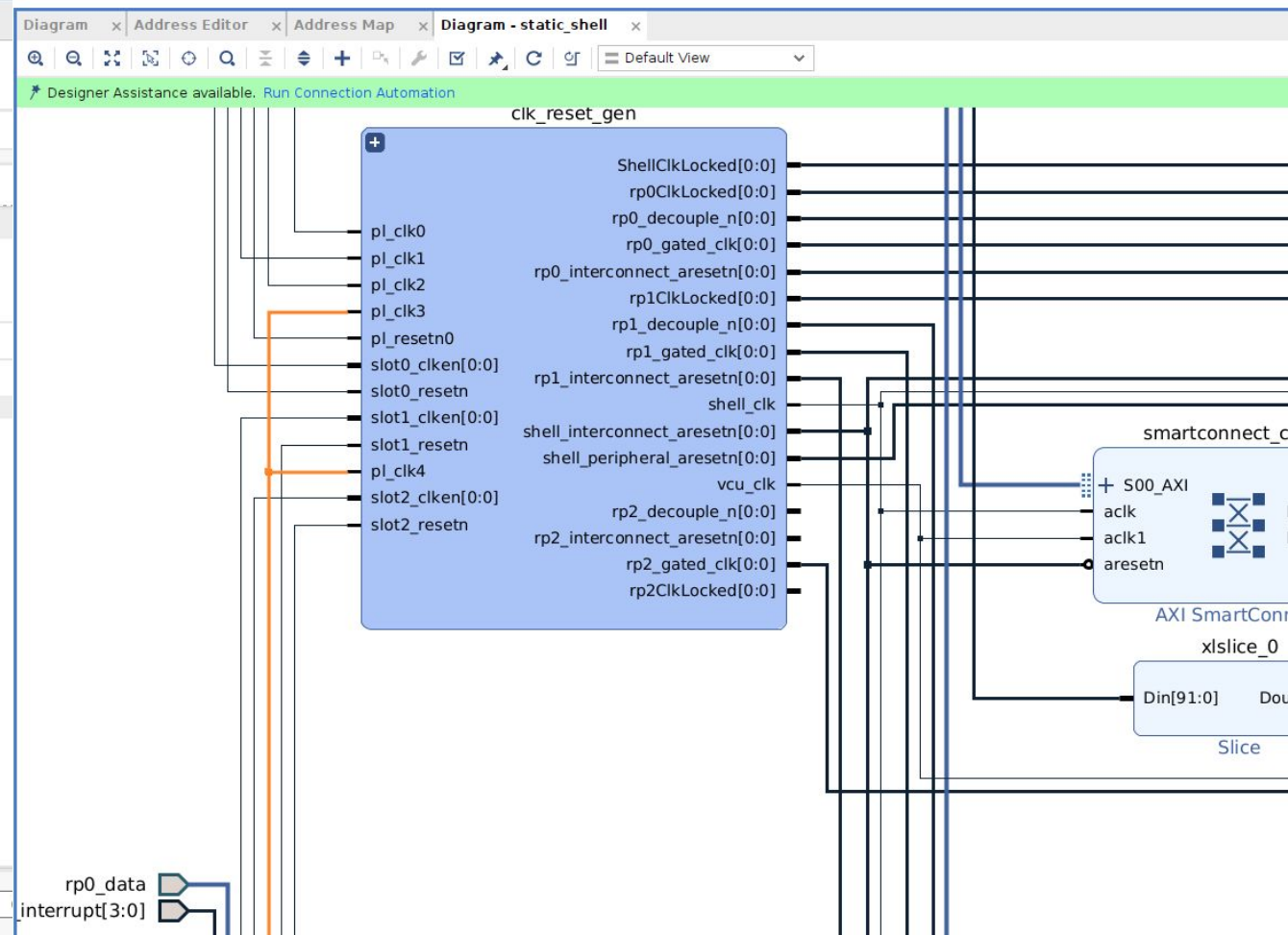
☐ Enable Manual Mode

> PLL Options

Search: Q:

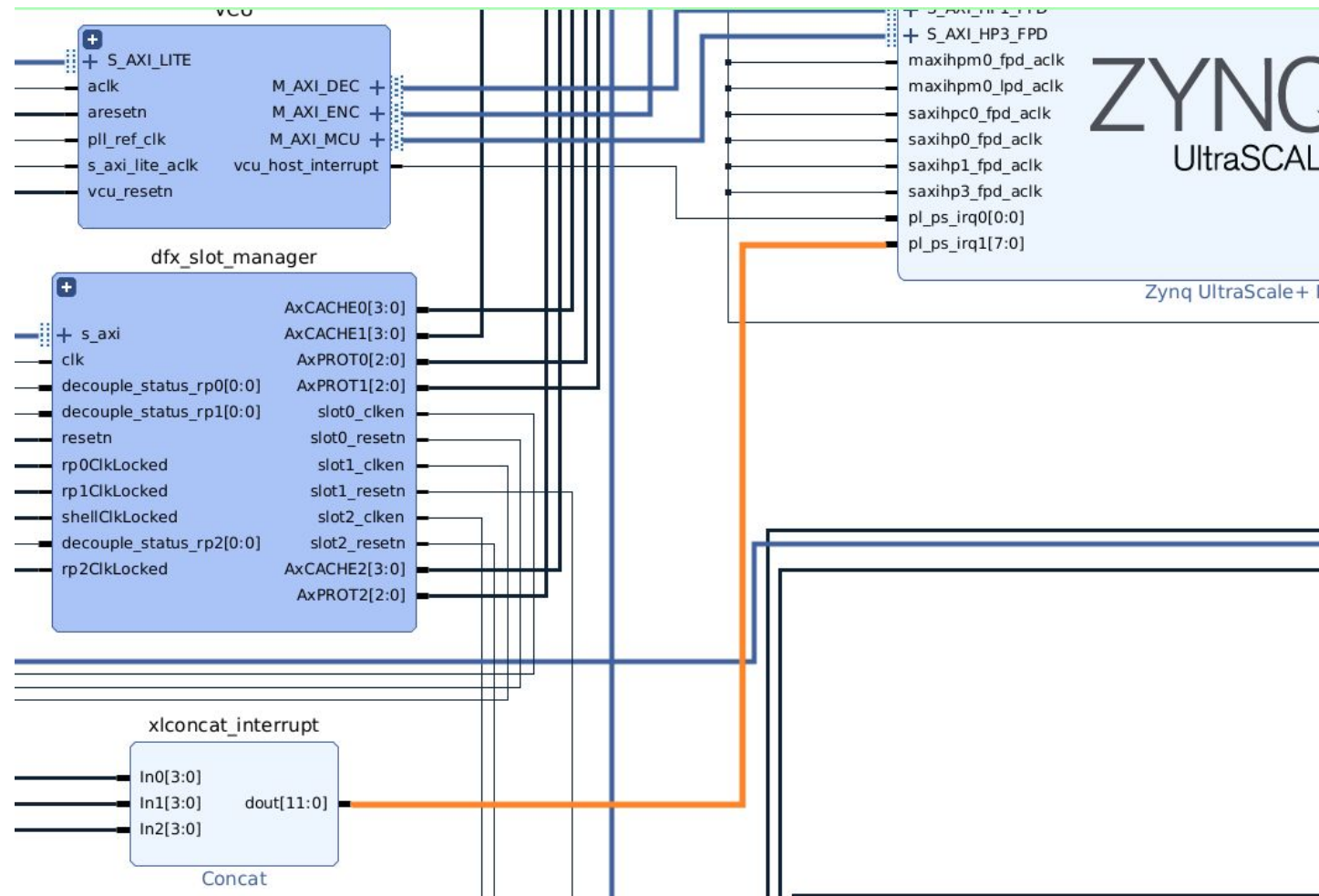
Name	Source	FracEn	Requested Freq (MHz)	Divisor 0	Divisor 1	Actual Frequency (MHz)	Range
Low Power Domain Clocks							
> Processor/Memory Clocks							
> Peripherals/I/O Clocks							
PL Fabric Clocks							
☒ PL0	IOPL		250	6	1	249.997498	0.000...
☒ PL1	RPLI		60	25	1	59.999401	0.000...
☒ PL2	RPLI		250	6	1	249.997498	0.000...
☒ PL3	RPLI		250	6	1	249.997498	0.000...
> System Debug Clocks							
> Full Power Domain Clocks							
> Advance Clocks							

OK



Implementation Notes

- PL_PS_IRQ1 interrupt bits are not enough. Delete VCU to use IRQ0.



Implementation Notes

- `write_bd_tcl ./opendfx.tcl` to generate the tcl script for the block design
 - `validate_bd_design` is not necessary, later steps will do this implicitly
 - **must `save_bd_design` after `validate_bd_design`, otherwise the synthesis will fail**
 - the IPI_PROP_COUNT mismatch: no documentation
 - it is incremented by 1 for every `write_bd_tcl`, so it may be a version mark

```
fjtcin@ustc:~/Documents/git/dfx-3rp (main)$ diff project_1/opendfx.tcl project_113c113
<      IPI_PROP_COUNT {10}\
---
>      IPI_PROP_COUNT {9}\
293c293
<      IPI_PROP_COUNT {10}\
---
>      IPI_PROP_COUNT {9}\
2609c2609
<  validate_bd_design
---
> #  validate_bd_design
```

Re-customize IP

DFX Decoupler (1.0)

Documentation IP Location

IP Symbol Validation Information

☐ Show disabled ports

Component Name: static_shell/shim_rp0/dfx_decoupler

Global Options Interface Options

Interface to Configure: config (xilinx.com:interface:aximm rtl:1.0)

Interface Name: config

Interface VLNW: xilinx.com:interface:aximm rtl:1.0

The Protocol overlay to apply to the VLNW: AXI4LITE

Interface Mode at RP: Slave

Number of CDC Stages: 0

☐ Add registers to interface (Please read documentation first)

Signal Options

New signal name:

Name	Management	Present	Direction at the RP	Width (bits)	Decouple
ARVALID	Auto	☒	As defined in the interface specification	1	☒
ARREADY	Auto	☒	As defined in the interface specification	1	☒
AWVALID	Auto	☒	As defined in the interface specification	1	☒
AWREADY	Auto	☒	As defined in the interface specification	1	☒
BVALID	Auto	☒	As defined in the interface specification	1	☒
BREADY	Auto	☒	As defined in the interface specification	1	☒
RVALID	Auto	☒	As defined in the interface specification	1	☒
RREADY	Auto	☒	As defined in the interface specification	1	☒
WVALID	Auto	☒	As defined in the interface specification	1	☒
WREADY	Auto	☒	As defined in the interface specification	1	☒

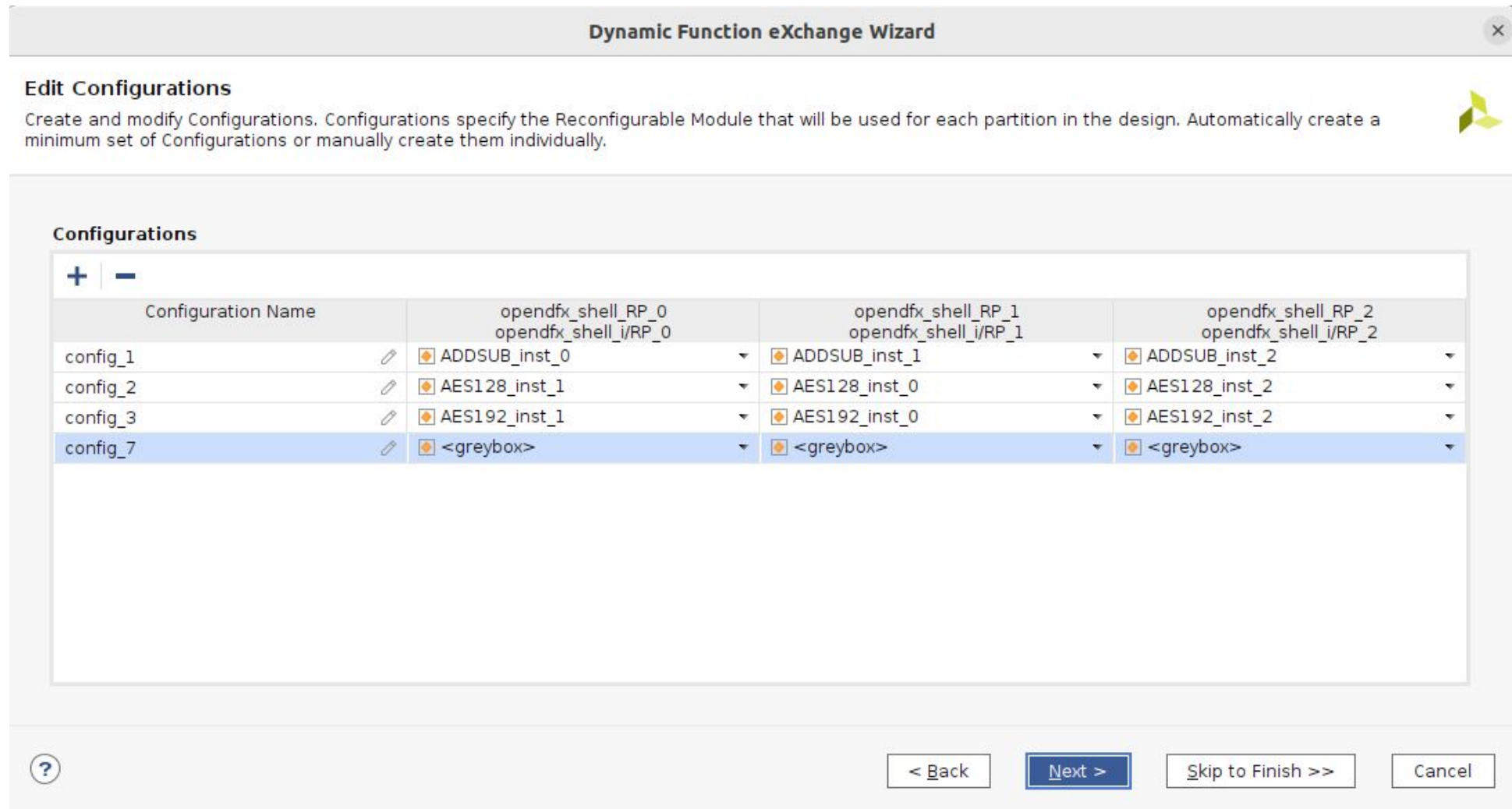
IP Symbol Validation Information

☒ s_config
☒ rp_data
☒ config_ack
☒ config_arstn
☒ data_ack
☒ data_arstn
☒ rp_sig_INTERRUPT[3:0]
☒ s_n_RST
☒ decouple

rp_config +
s_data +
rp_n_RST
decouple_status

Implementation Notes

- Configure DFX implementations: Using tcl commands / GUI
 - `report_property -all [get_pr_configurations config_7]`



Implementation Notes

- generate bitstreams of RMs
 - within the project for RP construction: monolithic
 - in separate projects, and link into the RP project checkpoints: modularized
 - preference for a monolithic or modularized design
 - for releasing, the monolithic way offers faster compilation and easier reproduction
 - for RM designing, choose the modularized way to design them one by one
- **the modularized design may not work for this project**
 - see the slide “Implementation Defects”

```
#Create DFX Configurations
setup_pr_configurations

launch_runs impl_1 -to_step write_bitstream -jobs 16
wait_on_run impl_1
# write_hw_platform -fixed -include_bit -force -file ./project_1/opendfx_shell_wrapper.xsa
exec cp ./project_1/project_1.runs/impl_1/opendfx_shell_wrapper.bit ./platform/
open_run impl_1
write_abstract_shell -force -cell opendfx_shell_i/RP_0 ./abstract_shell_RP_0.dcp
write_abstract_shell -force -cell opendfx_shell_i/RP_1 ./abstract_shell_RP_1.dcp
close_design
launch_runs child_1_impl_1 child_2_impl_1 child_3_impl_1 child_4_impl_1 child_5_impl_1 -to_step write_bitstream -jobs 16
wait_on_run child_1_impl_1
wait_on_run child_2_impl_1
wait_on_run child_3_impl_1
wait_on_run child_4_impl_1
wait_on_run child_5_impl_1
open_run impl_1
write_abstract_shell -force -cell opendfx_shell_i/RP_0 ./create_new_rm/abstract_shells/abstract_shell_RP_0.dcp
write_abstract_shell -force -cell opendfx_shell_i/RP_1 ./create_new_rm/abstract_shells/abstract_shell_RP_1.dcp
```

Implementation Notes

- shell.json: the “isolation slots” field: refer to the [SIHA documentation](#); remove it or modify it will cause RM loading failed, or stuck the system
- The pblocks of shim_RPs and RPs must match: shim_rp0 corresponds to RP_0, shim_rp1 corresponds to RP_1, otherwise it will cause bus errors

```
xdc > physical_constraints.xdc
15 create_pblock shim_rp_0
16 add_cells_to_pblock shim_rp_0 [get_cells -quiet [list opendfx_shell i/static_shell/shim_rp1]]
17 resize_pblock shim_rp_0 -add {SLICE_X0Y0:SLICE_X16Y119 DSP48E2_X0Y0:DSP48E2_X3Y47 IOB_X0Y0:IOB_X0Y104}
18
19 create_pblock shim_rp_1
20 add_cells_to_pblock shim_rp_1 [get_cells -quiet [list opendfx_shell i/static_shell/shim_rp0]]
21 resize_pblock shim_rp_1 -add {SLICE_X0Y120:SLICE_X16Y239 DSP48E2_X0Y48:DSP48E2_X3Y95 IOB_X0Y104:IOB_X0Y215}
22
23 set_property parent pblock_static [get_pblocks shim_rp_0]
24 set_property parent pblock_static [get_pblocks shim_rp_1]
25
26
27 create_pblock pblock_0
28 add_cells_to_pblock [get_pblocks pblock_0] [get_cells -quiet [list opendfx_shell i/RP_1]]
29 resize_pblock pblock_0 -add {SLICE_X17Y0:SLICE_X60Y119}
30 resize_pblock pblock_0 -add {BUFG_GT_X0Y0:BUFG_GT_X0Y47}
31 resize_pblock pblock_0 -add {BUFG_GT_SYNC_X0Y0:BUFG_GT_SYNC_X0Y29}
32 resize_pblock pblock_0 -add {DSP48E2_X4Y0:DSP48E2_X12Y47}
33 resize_pblock pblock_0 -add {GTHE4_CHANNEL_X0Y0:GTHE4_CHANNEL_X0Y7 GTHE4_COMMON_X0Y0:GTHE4_COMMON_X0Y7}
34 resize_pblock pblock_0 -add {IOB_X1Y0:IOB_X1Y103}
35 resize_pblock pblock_0 -add {PCIE40E4_X0Y0:PCIE40E4_X0Y1}
36 resize_pblock pblock_0 -add {RAMB18_X0Y0:RAMB18_X2Y47 RAMB36_X0Y0:RAMB36_X2Y23}
37 resize_pblock pblock_0 -add {URAM288_X0Y0:URAM288_X0Y31}
38 set_property CONTAIN_ROUTING 1 [get_pblocks pblock_0]
39 set_property EXCLUDE_PLACEMENT 1 [get_pblocks pblock_0]
40 set_property IS_SOFT FALSE [get_pblocks pblock_0]
41 set_property SNAPPING_MODE ON [get_pblocks pblock_0]
42
43 create_pblock pblock_1
44 add_cells_to_pblock [get_pblocks pblock_1] [get_cells -quiet [list opendfx_shell i/RP_0]]
45 resize_pblock pblock_1 -add {SLICE_X17Y120:SLICE_X60Y239}
46 resize_pblock pblock_1 -add {BUFG_GT_X0Y48:BUFG_GT_X0Y95}
47 resize_pblock pblock_1 -add {BUFG_GT_SYNC_X0Y30:BUFG_GT_SYNC_X0Y59}
48 resize_pblock pblock_1 -add {CFGIO_SITE_X0Y0:CFGIO_SITE_X0Y0 CONFIG_SITE_X0Y0:CONFIG_SITE_X0Y0}
49 resize_pblock pblock_1 -add {DSP48E2_X4Y48:DSP48E2_X12Y95}
50 resize_pblock pblock_1 -add {GTHE4_CHANNEL_X0Y8:GTHE4_CHANNEL_X0Y15 GTHE4_COMMON_X0Y2:GTHE4_COMMON_X0Y2}
51 resize_pblock pblock_1 -add {IOB_X1Y104:IOB_X1Y207}
52 resize_pblock pblock_1 -add {RAMB18_X0Y48:RAMB18_X2Y95 RAMB36_X0Y24:RAMB36_X2Y47}
53 resize_pblock pblock_1 -add {SYSMONE4_X0Y0:SYSMONE4_X0Y0}
54 resize_pblock pblock_1 -add {URAM288_X0Y32:URAM288_X0Y63}
55 set_property CONTAIN_ROUTING 1 [get_pblocks pblock_1]
```

correct

Switch Between RP Designs

- Unload base shell firmware/bitstream using *sudo xmutil unloadapp base* before loading a flat accelerator such as k26-starter-kits ([source](#))
- Also unload base shell before loading another RP design (i.e. 2rp -> 3rp)
- This unload is mandatory: otherwise designs cannot be properly loaded

```
ubuntu@kria:~$ sudo xmutil loadapp FFT
FFT: Loaded with slot_handle 0
ubuntu@kria:~$ sudo xmutil unloadapp 0
remove from slot 0 returns: 0 (Ok)
ubuntu@kria:~$ sudo xmutil unloadapp base
remove from slot base returns: 0 (Ok)
ubuntu@kria:~$ sudo xmutil loadapp k26-starter-kits
k26-starter-kits: Loaded with slot_handle 0
ubuntu@kria:~$ sudo xmutil unloadapp 0
remove from slot 0 returns: 0 (Ok)
ubuntu@kria:~$ sudo xmutil loadapp FFT
FFT: Loaded with slot_handle 0
```

```
ubuntu@kria:~$ ls /lib/firmware/
6.8.0-1009-xilinx  ap1302_ar0144_single_fw.bin  ap1302_ar1335_single_fw.bin  regulatory.db  regulatory.db.p7s  xilinx
ubuntu@kria:~$ sudo xmutil loadapp FFT
FFT: Loaded with slot_handle 0
ubuntu@kria:~$ ls /lib/firmware/
6.8.0-1009-xilinx  ap1302_ar0144_single_fw.bin  ap1302_ar1335_single_fw.bin  opendfx_shell_i_RP_0_FFT_4channel_inst_0_partial.dtbo  opendfx_shell_wrapper.dtbo  regulatory.db  regulatory.db.p7s  xilinx
ubuntu@kria:~$ sudo xmutil unloadapp FFT
remove from slot FFT returns: 0 (Ok)
ubuntu@kria:~$ ls /lib/firmware/
6.8.0-1009-xilinx  ap1302_ar0144_single_fw.bin  ap1302_ar1335_single_fw.bin  opendfx_shell_wrapper.dtbo  regulatory.db  regulatory.db.p7s  xilinx
ubuntu@kria:~$ sudo xmutil unloadapp base
remove from slot base returns: 0 (Ok)
ubuntu@kria:~$ ls /lib/firmware/
6.8.0-1009-xilinx  ap1302_ar0144_single_fw.bin  ap1302_ar1335_single_fw.bin  regulatory.db  regulatory.db.p7s  xilinx
ubuntu@kria:~$ sudo xmutil loadapp k26-starter-kits
k26-starter-kits: Loaded with slot_handle 0
ubuntu@kria:~$ ls /lib/firmware/
6.8.0-1009-xilinx  ap1302_ar0144_single_fw.bin  ap1302_ar1335_single_fw.bin  k26_starter_kits.dtbo  regulatory.db  regulatory.db.p7s  xilinx
```

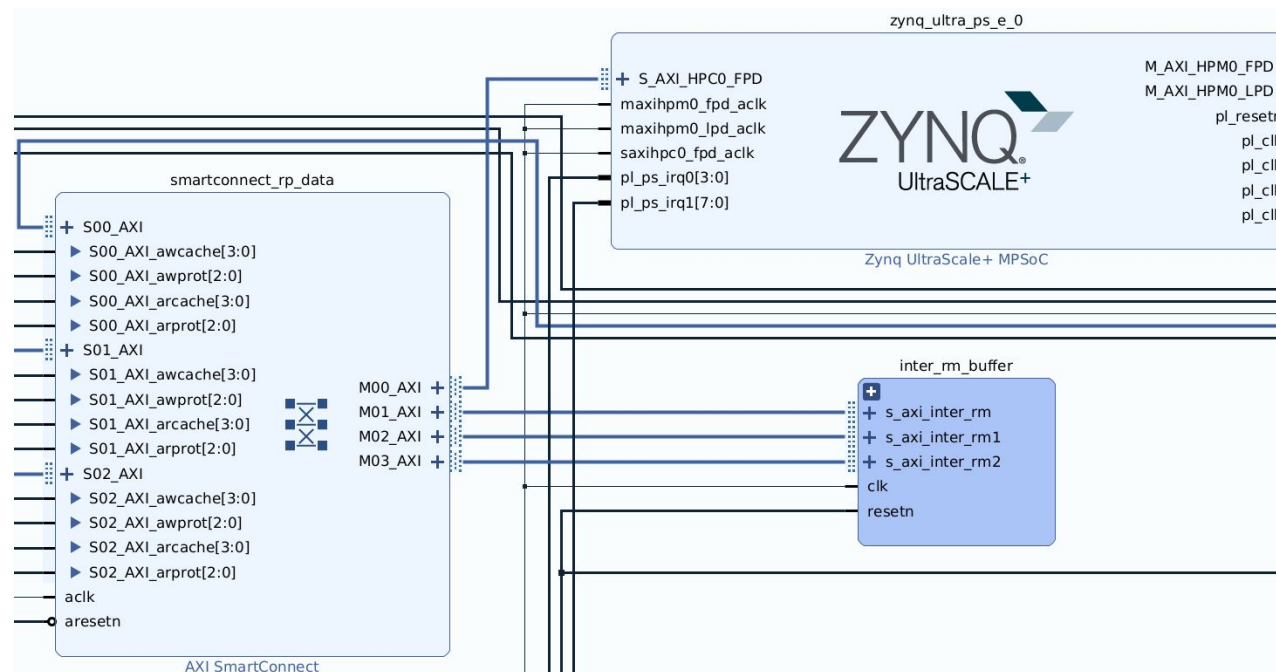

Implementation Defects

- RP2 cannot be unloaded if it is the last active RP
 - nothing to do with pblock partitions; may be a bug of xmutil
- the modularized design can cause bus errors
 - no such problem on another KV260 board; may be a bug from Xilinx
- *Load* and *Run* the reference RM_2 module before running other RM_2, otherwise the system may stuck (do this for every newly loaded base)

```
ubuntu@kria:~/Documents/apps$ sudo xmutil loadapp TWICE
TWICE: loaded to slot 0
ubuntu@kria:~/Documents/apps$ sudo xmutil loadapp TWICE
TWICE: loaded to slot 1
ubuntu@kria:~/Documents/apps$ sudo xmutil loadapp TWICE
TWICE: loaded to slot 2
ubuntu@kria:~/Documents/apps$ sudo ./testTWICE 2
Success: Selected Operation Done !.
2.000000 4.000000 6.000000 8.000000 2.000000 4.000000 6.000000
ubuntu@kria:~/Documents/apps$ sudo xmutil unloadapp 2
remove from slot 2 returns: 0 (Ok)
ubuntu@kria:~/Documents/apps$ sudo xmutil unloadapp 1
remove from slot 1 returns: 0 (Ok)
ubuntu@kria:~/Documents/apps$ sudo xmutil unloadapp 0
remove from slot 0 returns: 0 (Ok)
ubuntu@kria:~/Documents/apps$ sudo xmutil loadapp FOUR
FOUR: loaded to slot 0
ubuntu@kria:~/Documents/apps$ sudo xmutil loadapp FOUR
FOUR: loaded to slot 1
ubuntu@kria:~/Documents/apps$ sudo xmutil loadapp FOUR
FOUR: loaded to slot 2
ubuntu@kria:~/Documents/apps$ sudo ./testTWICE 2
Success: Selected Operation Done !.
4.000000 8.000000 12.000000 16.000000 4.000000 8.000000 12.000000
```

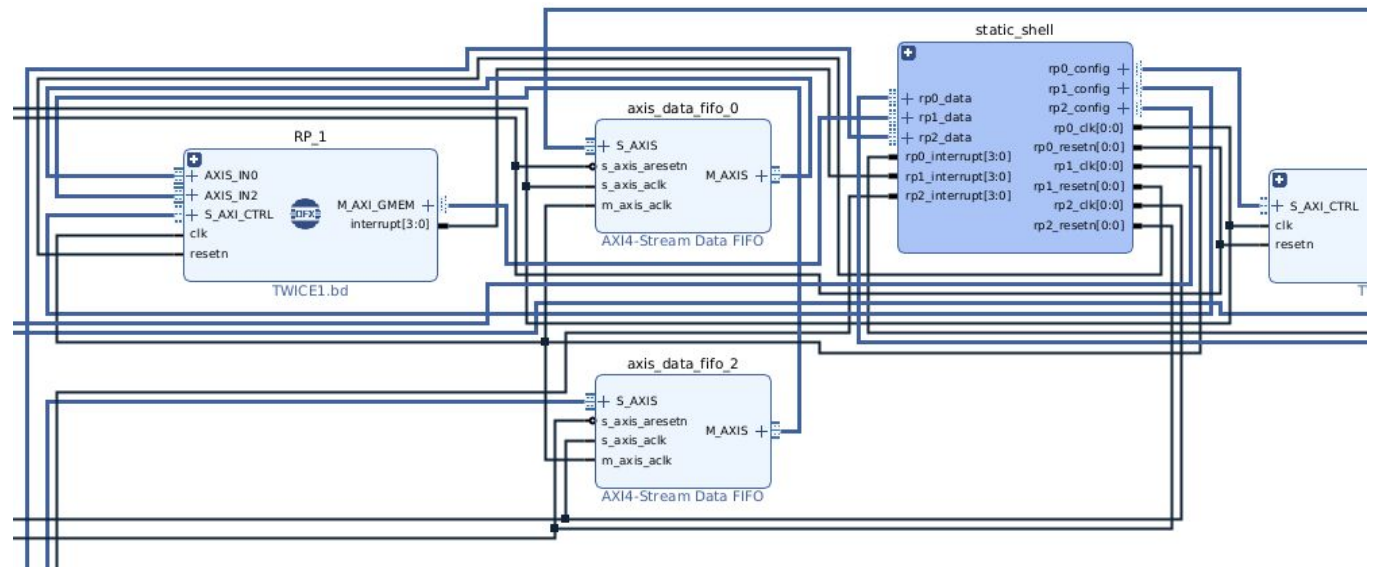
SPMM: Methods

- decode-execute-encode architecture (RP_0 -> RP_2 -> RP_1)
 - decode the sparse format to dense, execute GEMM, and encode the result
 - GEMM implementation: tiled systolic array
 - RMs can communicate with each other by sharing the DDR buffers ([source](#))
 - DDR -> Buffer -> AXIMM -> AXIS -> Accelerator -> AXIS -> AXIMM -> Buffer -> DDR
 - rm_comm_box provides high-bandwidth DMA between the AXI4 and AXI4-Stream
 - However, inter_rm_buffer is not utilized (it cannot be used in the current AXIS framework)



AxIS Inter-RM Communication

- AXI4 Streams can be pipelined, so AXIS is preferred over AXIMM
- AXI4 Streams directly connecting RMs ([AXI4-Stream Clock Converter](#) IP)
- AXI4 Streams connecting with a dual-FIFO ([AXI4-Stream Data FIFO](#) IP)
- custom HLS implementation of multiplexers ([ap_ctrl_none](#) protocol)
- reconfigure the frequency of RP_clocks (may “bus error” if timing fails)
- RP_0
 - Input: Memory
 - Output: Memory, RP_1, RP_2
- RP_1
 - Input: Memory, RP_0, RP_2
 - Output: Memory
- RP_2
 - Input: Memory, RP_0
 - Output: Memory, RP_1



Background: AXI Control Interfaces

Vitis RTL Kernel Standard Execution Model

ap_ctrl_none

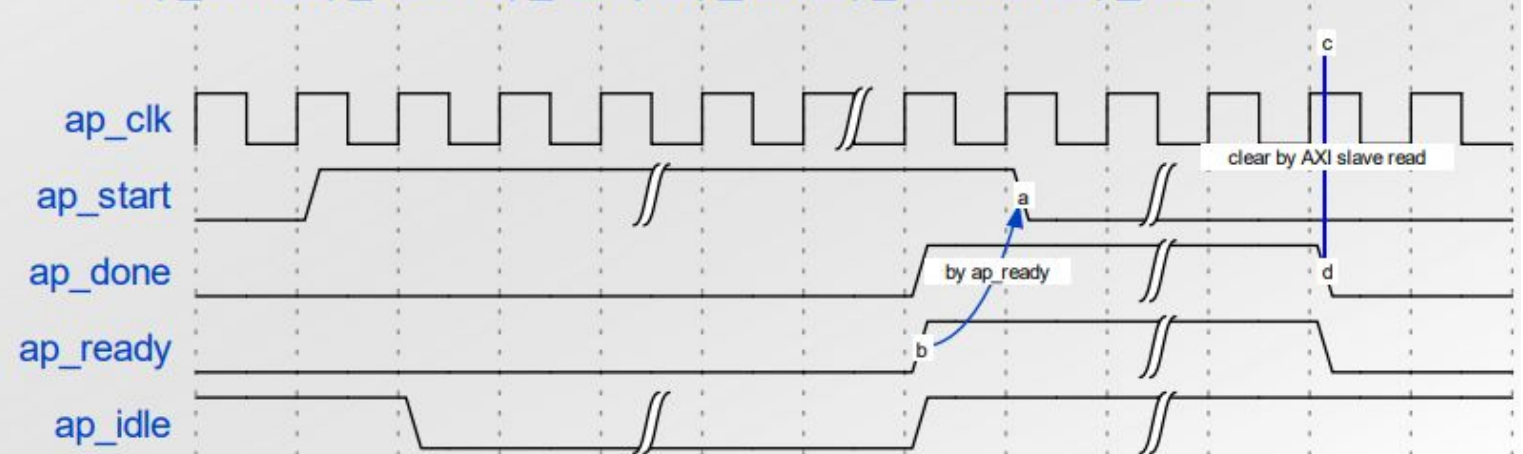
- No host control
- Free-running

ap_ctrl_hs

- Controlled via AXI slave
- Hand-shake protocol
- Sequential kernel
- Input/output coupled

AXI slave register signals:

ap_start, *ap_done*, *ap_ready*, *ap_idle*, *ap_continue*, *ap_idle*



ap_ctrl_chain

- Controlled via AXI slave
- Extension to *ap_ctrl_hs*
- Pipelined kernel
- Input/output decoupled

